RVC Dataset Studio

Plano de evolução técnica, UI/UX e qualidade — paridade funcional desktop/mobile

Documento de implementação | Revisão do repositório `main` em 10 de outubro de 2026 Objetivo: definir o que melhorar, como implementar, porquê, como validar e em que ordem. **Requisito transversal:** mobile deve disponibilizar todas as funções do desktop; só o layout e os métodos de interação se adaptam. **Âmbito:** integridade do áudio exportado, armazenamento local, resiliência, testes, UI/UX e experiência funcional completa em desktop e mobile, além da apresentação no GitHub.

> **Recomendação central:** não reescrever a aplicação. Preservar a arquitetura atual e corrigir primeiro os pontos que afetam a confiança nos dados; em seguida, melhorar a recuperação de falhas e o fluxo de gravação; por fim, investir na experiência visual, na acessibilidade e na apresentação pública.

1. Resumo executivo

O RVC Dataset Studio tem uma proposta clara: orientar a criação de gravações vocais e preparar datasets para Applio/RVC diretamente no navegador, com armazenamento local. O repositório já contém uma base técnica útil — Preact, TypeScript, Web Audio API/AudioWorklet, IndexedDB, exportação WAV/ZIP, testes Vitest e o comando agregado npm run check.

A evolução recomendada deve seguir quatro objetivos:

1. **Integridade dos dados:** o áudio, os metadados e as recomendações de treino devem concordar entre si.
2. **Nenhuma perda silenciosa:** o utilizador deve saber quando uma gravação está guardada, apenas em memória ou com falha de persistência.
3. **Escala e resiliência:** exportar sessões grandes sem criar cópias desnecessárias de todos os WAV na memória e recuperar estados de erro.
4. **Fluxo de estúdio coerente em todos os ecrãs:** permitir gravar, rever, aceitar/repetir e acompanhar a cobertura do dataset com o mínimo de distrações, sem retirar funções no telemóvel.
5. **Paridade funcional total:** toda a função disponível no desktop deve ter uma forma equivalente de acesso e utilização em mobile. Muda a disposição dos controlos, não as capacidades do produto.

Ordem de prioridade

Prioridade	Significado	Exemplos
P0 — Integridade	Corrigir antes de promover a aplicação como ferramenta fiável de produção	Frequência de amostragem coerente; confirmação de gravação em IndexedDB; erros de exportação visíveis
P1 — Robustez	Reduzir falhas e suportar sessões maiores	Exportação por blocos; backup/restauro; testes de navegador
P2 — Produto e UX	Tornar a utilização mais clara, rápida e profissional	Dashboard de cobertura; revisão de takes melhorada; onboarding e acessibilidade
P3 — Apresentação	Aumentar a compreensão e a adoção do projeto	Capturas reais, vídeo curto, README mais direto, releases e changelog

2. Âmbito e limites da revisão

Esta documentação resulta de uma **revisão estática do código e da documentação públicos** do repositório. Não foi executado um ciclo completo de gravação em hardware real nem uma bateria de testes em Chrome, Firefox, Safari e dispositivos móveis. Assim:

- As referências a ficheiros e funções existentes são observações do código consultado.
- As propostas são recomendações de implementação, não funcionalidades já existentes.
- A prioridade final pode mudar depois de executar npm ci e npm run check, medir consumo de memória e testar o fluxo completo nos navegadores suportados.
- Não se recomenda substituir toda a stack, mudar de framework ou redesenhar tudo de uma só vez.

Base técnica observada

- **Interface:** Preact e @preact/signals.
- **Linguagem/build:** TypeScript e Vite.
- **Áudio:** Web Audio API e captura baseada em AudioWorklet.
- **Dados locais:** IndexedDB para takes e localStorage para determinadas definições/conteúdos personalizados.
- **Exportação:** codificação WAV e biblioteca fflate para ZIP.
- **Validação:** ESLint, TypeScript, validação de conteúdo, Vitest e build, agregados em npm run check.
- **Conteúdo:** exercícios e packs linguísticos validados por schemas.

A abordagem certa é, portanto, **evoluir por pequenas alterações testáveis**, reutilizando os módulos existentes de áudio, análise, armazenamento, exportação e UI. A paridade funcional mobile–desktop é um requisito de produto a implementar e validar; esta revisão não afirma que a aplicação já a cumpre integralmente.

---

3. Diagnóstico: o que corrigir primeiro

ID	Observação no código/documentação	Risco prático	Prioridade
D-01	applioDatasetInfo() declara a frequência real em audio_format.sample_rate, mas fixa applio_recommended_settings.sample_rate em 40k. Cada WAV é codificado com a frequência do respetivo take.	O manifesto pode recomendar uma frequência diferente da que está nos ficheiros, criando ambiguidade no passo de preparação do Applio.	P0
D-02	datasetEntries() codifica cada take num Uint8Array WAV e reúne todas as entradas antes da montagem do ZIP. buildZip() recolhe os blocos emitidos até concluir o ficheiro.	O pico de memória pode crescer bastante em datasets grandes, porque os PCM originais, os WAV codificados e os blocos do ZIP coexistem temporariamente.	P0/P1
D-03	accept() atualiza a interface e avança o exercício antes de confirmar o resultado de putTake(). Se a persistência falhar, aparece um aviso, mas o take já foi aceite no estado em memória.	O utilizador pode interpretar "aceite" como "guardado de forma duradoura" e perder o take ao atualizar/fechar a página.	P0
D-04	measureNoise() e runMicCheck() iniciam captura e aguardam vários segundos sem um try/finally abrangente.	Uma exceção durante captura/paragem pode deixar a interface presa em "em curso" ou impedir a limpeza correta do estado.	P0/P1

ID	Observação no código/documentação	Risco prático	Prioridade
D-05	downloadZip() não envolve toda a preparação/exportação num tratamento de erro próprio.	Uma falha de encoding ou ZIP pode não dar uma mensagem recuperável e clara ao utilizador.	P1
D-06	requestPersistence() devolve um booleano, mas boot() não comunica visualmente o resultado ao utilizador.	A aplicação não explica se o navegador aceitou o pedido de armazenamento persistente. Isto não significa, por si só, que os dados se percam.	P1
D-07	A exportação de WAV contínuo seleciona os takes com a mesma frequência do primeiro e comunica quando outros são ignorados.	O utilizador pode não perceber quais os takes que ficaram de fora sem uma lista explícita antes de descarregar.	P1
D-08	O README dá recomendações de treino e de distribuição vocal bastante específicas, embora sejam dependentes de dados, modelo e configuração. Também apresenta limites de ruído em contextos distintos sem definir claramente “aceitável” versus “ideal”.	Algumas heurísticas podem ser interpretadas como garantias técnicas universais.	P1
D-09	A documentação de exercícios contém a sigla SOVPE; a designação comum é **SOVT** (*Semi-Occluded Vocal Tract*).	Pequena falha editorial que reduz a confiança técnica e pode confundir quem procura fontes externas.	P2

O que manter

Manteria a stack atual, o processamento local como princípio central, o modelo modular do código, o CI existente e os testes com sinais sintéticos. Estas bases reduzem o custo de implementação e permitem investir o esforço no comportamento real do produto.

---

4. Plano técnico detalhado

T-01 — Tornar a frequência de amostragem consistente

Problema: os WAV usam a frequência do take, enquanto o manifesto de treino recomenda sempre 40k. O código distingue a frequência do áudio (audio_format.sample_rate) da recomendação de treino, mas não deixa explícito se o áudio será ou não reamostrado.

Como implementar:

1. Introduzir uma definição única para a política de frequência de exportação: preserve-source, 40000 ou 48000.
2. Mostrar a frequência de origem detetada e a frequência final escolhida antes de exportar.
3. Se o utilizador escolher uma frequência diferente da origem, reamostrar efetivamente o sinal com um método adequado, incluindo filtragem anti-aliasing quando necessário. Não alterar apenas o valor do cabeçalho WAV.
4. Gerar todos os WAV do pacote com a frequência de destino escolhida, ou preservar cada frequência e declarar claramente a mistura no manifesto, se esse modo for deliberadamente suportado.
5. Guardar no dataset_info.json, pelo menos, source_sample_rates, output_sample_rate, resampled e a política utilizada. Adaptar o esquema se existir validação formal desse JSON.
6. Tornar a recomendação do Applio explicitamente uma recomendação, não uma descrição enganadora do áudio realmente exportado.

7. Na exportação de WAV contínuo, ou converter todos os takes para uma frequência comum, ou apresentar uma seleção explícita dos takes incompatíveis antes de exportar.

Porquê: um dataset deve ser interpretável sem adivinhação. A frequência declarada, o cabeçalho dos WAV e as opções de preparação devem concordar.

Testes de aceitação:

- Criar takes sintéticos a 40 kHz, 44,1 kHz e 48 kHz.
- Abrir e inspecionar os cabeçalhos WAV dos ficheiros exportados.
- Confirmar que a frequência do cabeçalho corresponde à frequência de destino declarada no JSON.
- Confirmar que o reamostramento produz o número de amostras esperado e não gera NaN, clipping artificial ou ficheiros vazios.
- Confirmar que o modo de preservação declara corretamente as frequências existentes.
- Testar ZIP com vários takes e confirmar que o conteúdo continua legível para ferramentas de inspeção ZIP/WAV.

Definição de concluído: não existe nenhuma combinação em que o manifesto diga que os WAV estão a 40 kHz quando os ficheiros exportados estão a outra frequência sem essa diferença estar explícita.

T-02 — Distinguir “aceite” de “guardado”

Problema: o estado visível e o avanço do exercício ocorrem antes de a escrita no IndexedDB estar confirmada.

Como implementar:

1. Modelar o estado de persistência de cada take: saving, saved, unsaved e save-error.
2. Ao carregar em **Aceitar take**, iniciar a escrita e aguardar o resultado, ou manter o take visível com um indicador persistente de “a guardar” até receber confirmação.
3. Se a escrita falhar, manter o take recuperável no estado atual e oferecer **Tentar novamente**, **Exportar take** e, apenas se o utilizador decidir, **Continuar sem guardar**.
4. Não apresentar o take como guardado apenas porque foi adicionado ao estado em memória.
5. Ao mudar de exercício ou fechar uma vista, não apagar silenciosamente um take não guardado; pedir uma decisão quando houver risco real de perda.
6. Registar métricas locais de diagnóstico apenas se forem inteiramente locais; não introduzir telemetria de rede sem uma decisão de privacidade separada.

Porquê: a confiança no estúdio depende tanto da captura como da certeza de que o trabalho sobrevive a um refresh.

Testes de aceitação:

- Simular putTake() a devolver false.
- Confirmar que o utilizador vê um estado de falha e consegue tentar novamente.
- Confirmar que o estado não avança silenciosamente como se o take estivesse seguro.
- Atualizar a página depois de uma gravação confirmada e verificar a recuperação.
- Testar o modo em que IndexedDB está indisponível.

Definição de concluído: cada take aceite tem um estado de persistência explícito e a falha de armazenamento nunca é silenciosa.

T-03 — Tornar as operações assíncronas recuperáveis

Escopo inicial: medição de ruído, verificação do microfone, análise de voz, gravação, paragem de captura e exportação ZIP.

Como implementar:

1. Envolver as operações que mudam o estado da interface em try/catch/finally.
2. Garantir no finally que flags como busy, measuring, capturing e stopping regressam a um estado válido.
3. Centralizar a apresentação dos erros de domínio — por exemplo, permissão recusada, microfone removido, formato inválido, armazenamento cheio ou falha de exportação — sem apagar os detalhes técnicos necessários ao diagnóstico.
4. Introduzir cancelamento controlado nas operações longas, com limpeza de AudioWorklet, AudioContext, temporizadores e buffers temporários quando apropriado.

5. Tratar a perda do microfone a meio da gravação como um estado de sessão recuperável: preservar o que tiver sido capturado e explicar se existe um take parcial.
6. Não usar mensagens genéricas como “algo correu mal” quando for possível indicar uma ação de recuperação.

Porquê: exceções em áudio e armazenamento são normais em navegadores; a UI tem de regressar a um estado utilizável mesmo quando uma operação falha.

Testes de aceitação: injetar falhas nas etapas de iniciar captura, parar captura, analisar, escrever no IndexedDB e construir o ZIP; em todas, confirmar que os indicadores ficam coerentes e que o utilizador recebe uma ação concreta.

T-04 — Reduzir a memória usada na exportação

Problema: a exportação transforma primeiro todos os takes em WAV completos. Depois, o ZIP é construído e os blocos de saída permanecem na memória até ao download.

Como implementar por etapas:

1. Medir um baseline com, pelo menos, um dataset curto, um médio e um grande. Registar duração de áudio, bytes exportados, tempo e comportamento do navegador.
2. Separar a codificação WAV numa API que possa emitir pedaços do cabeçalho e das amostras sem ter de criar um Uint8Array completo para cada take.
3. Usar a API de ZIP por blocos para enviar os segmentos progressivamente, evitando uma lista que retenha todos os WAV já codificados.
4. Quando suportado, ligar a geração a uma escrita progressiva em ficheiro. Manter fallback de Blob/download para navegadores sem a API escolhida.
5. Mostrar progresso por ficheiros e bytes estimados, botão para cancelar e resultado final explícito.
6. Libertar buffers temporários após o uso, sem apagar os PCM que continuam a ser necessários para editar ou voltar a exportar o projeto.
7. Definir um comportamento claro para memória insuficiente: não limpar takes nem marcar a exportação como concluída.

Porquê: a aplicação é local, mas isso não significa memória ilimitada. A exportação é o ponto em que os dados de áudio podem ser duplicados.

Testes de aceitação: comparar antes/depois o pico de memória e a taxa de sucesso em dispositivos de referência; confirmar que o ZIP abre e contém o número esperado de ficheiros; interromper a exportação a meio e confirmar que a sessão original continua intacta.

Definição de concluído: o percurso de exportação não cria intencionalmente uma cópia WAV completa de todos os takes em simultâneo. O valor máximo aceitável de memória deve ser definido após a medição do baseline, não inventado antecipadamente.

T-05 — Armazenamento, capacidade e backup/restauro

Como implementar:

1. Mostrar um indicador de armazenamento: estado da base de dados, número de takes, espaço utilizado quando a API o disponibiliza e resultado do pedido de persistência do navegador.
2. Explicar claramente que o armazenamento local depende do browser, do perfil e das políticas do dispositivo; persistência solicitada não equivale a uma garantia absoluta contra perda.
3. Criar um **backup de projeto** separado do pacote final para treino. O backup deve incluir takes, definições relevantes, perfil, seleção de exercícios, linhas personalizadas e versão do formato de backup.
4. Implementar importação validada: verificar versão, tamanho, estrutura, nomes e formatos antes de inserir dados na base local.
5. Importar de forma transacional ou preparar primeiro os dados e só depois confirmar a substituição/mesclagem.
6. Mostrar resumo antes do restauro: quantos takes serão adicionados, substituídos ou ignorados.
7. Pedir confirmação extra para apagar todo o projeto ou substituir dados existentes.

Porquê: o ZIP de treino é um artefacto final; não substitui um backup de trabalho capaz de recuperar o estado de edição.

Testes: backup seguido de restauro; ficheiro corrompido; versão futura desconhecida; importação duplicada; falta de espaço; cancelamento antes da confirmação; tentativa de apagar o projeto durante uma gravação.

T-06 — Tornar os parâmetros de treino transparentes

Problema: o dataset_info.json inclui valores específicos, como 40k, batch size, épocas e recomendação de modelo pré-treinado. Esses valores podem depender da versão do Applio/RVC, do modelo base e do hardware.

Como implementar:

1. Separar "informação do áudio" de "sugestões de treino".
2. Marcar os parâmetros como **sugestões iniciais** e indicar a data/versão de referência da recomendação.
3. Tornar configuráveis apenas os valores que o projeto consegue explicar e validar; evitar construir uma falsa promessa de compatibilidade universal.
4. Sempre que possível, oferecer um perfil conservador e deixar o resto avançado/opcional.
5. Rever as recomendações com testes documentados em versões identificadas do Applio; não declarar que uma determinada distribuição vocal evita artefactos sem evidência comparativa.
6. Corrigir a referência editorial de SOVPE para SOVT e definir os limites de ruído como níveis "ideal", "aceitável" e "recomenda-se repetir", sem misturar conceitos.

Porquê: metadados úteis devem reduzir o trabalho manual sem criar uma falsa sensação de certeza.

---

5. Plano de UI/UX

5.1 Princípios de experiência

- **Uma ação primária por ecrã:** gravar, aceitar, repetir ou exportar, consoante o estado.
- **Estado sempre visível:** pronto, a preparar, a gravar, a analisar, a guardar, guardado ou com erro.
- **Recomendações explicáveis:** cada aviso de qualidade deve dizer o que foi detetado, o impacto provável e o que fazer a seguir.
- **Progressive disclosure:** mostrar primeiro o essencial; manter opções técnicas avançadas fechadas até serem pedidas.
- **Controlo do utilizador:** um aviso de qualidade não deve apagar dados nem forçar uma repetição automaticamente.
- **Privacidade compreensível:** distinguir dados guardados no browser, pacote de treino exportado e backup do projeto.
- **Acessibilidade desde o início:** teclado, foco visível, contraste, leitores de ecrã, alvos de toque e movimento reduzido.
- **Paridade funcional, não paridade de layout:** no mobile, os mesmos recursos, dados, definições, análises, controlos e formatos de exportação estão disponíveis; apenas a organização visual e os métodos de interação se adaptam ao ecrã e ao sistema operativo.

5.2 Navegação proposta

Organizar o fluxo mentalmente em cinco áreas, mantendo componentes e rotas simples se a aplicação atual não precisar de um router completo:

1. **Visão geral** — estado do projeto, próxima ação e resumo da sessão.
2. **Perfil vocal** — teste guiado, intervalo detetado e ajuste manual.
3. **Estúdio** — exercício atual, contador, controlo grande de gravação e guia de pitch.
4. **Takes** — lista, reprodução, qualidade, guardar/voltar a gravar e estado de persistência.
5. **Dataset e exportação** — cobertura, distribuição, formatos, validações, backup e exportação.

As definições globais (microfone, idioma, guia de áudio, atalhos, formato de exportação e opções avançadas) ficam num local consistente. Desktop e mobile expõem as mesmas cinco áreas e os mesmos comandos. Em ecrãs estreitos, painéis

lado a lado podem transformar-se em separadores, acordeões ou folhas de detalhe, sem esconder definitivamente funcionalidades. A prioridade é tornar a hierarquia previsível, não reduzir o produto.

5.3 Fluxo de primeira utilização

Ecrã 1 — Introdução Explicar em poucas linhas: gravação local; necessidade de permitir o microfone; possibilidade de exportar para Applio/RVC; consentimento para gravar apenas voz própria ou com autorização.

Ecrã 2 — Preparar microfone Selecionar dispositivo quando possível, pedir permissão só quando necessário e mostrar instruções adequadas ao erro. Distinguir “permissão recusada”, “dispositivo ausente” e “navegador incompatível”.

Ecrã 3 — Teste de nível Mostrar um medidor de pico e um estado em linguagem simples: “nível baixo”, “bom nível” ou “possível clipping”. Não exigir um ambiente profissional; explicar como corrigir a distância/gain.

Ecrã 4 — Perfil vocal opcional Permitir realizar a análise ou avançar com intervalo manual/predefinido. Apresentar o resultado como estimativa e permitir ajuste posterior.

Ecrã 5 — Primeira gravação Mostrar um único botão primário, um contador grande, a instrução vocal e atalhos. Uma indicação breve deve esclarecer quando começa a gravação real.

5.4 Ecrã de gravação: hierarquia visual

A proposta para o ecrã principal é:

- **Topo:** nome da sessão, progresso global e estado do microfone.
- **Centro:** texto/instrução do exercício, com destaque para a sílaba ou frase atual.
- **Ação primária:** botão grande “Gravar”/“Parar”, com contador e estado de gravação visível.
- **Monitorização:** indicação de pitch e nível, sem dominar a leitura do texto.
- **Rodapé:** exercício anterior/seguinte, opção de repetir e atalhos disponíveis.
- **Detalhes avançados:** guia de pitch, pré-contagem, threshold VAD e outras opções num painel recolhido.

Durante a gravação, evitar mudanças de layout, notificações que tapem o texto, animações excessivas ou botões pequenos. O estado “A GRAVAR” deve ser comunicado por texto e forma visual, não apenas por cor.

5.5 Revisão de take: tornar a decisão óbvia

Cada take deve ter um cartão consistente com:

- Reprodução e waveform/forma de onda, se disponível sem custos excessivos.
- Duração total e duração vocal útil.
- Pico e avaliação de ruído, quando medidos.
- Estado de qualidade com três níveis: **Bom**, **Rever** ou **Repetir recomendado**.
- Explicação de cada alerta: “pico demasiado alto”, “pouca voz detetada”, “nível baixo” ou “possível ruído”.
- Ações grandes e previsíveis: **Aceitar**, **Gravar novamente**, **Ouvir outra vez**.
- Estado de persistência independente da qualidade: “A guardar…”, “Guardado localmente”, “Não guardado — tentar novamente”.

Não transformar o QC numa nota inexplicável. O objetivo é apoiar uma decisão informada, não afirmar que um algoritmo sabe sempre se uma gravação é boa para treino.

5.6 Dashboard de qualidade do dataset

Criar uma visão agregada com os indicadores seguintes:

- Número de takes gravados, aceites, pendentes e com falha de persistência.
- Minutos totais e minutos vocais úteis.
- Cobertura de notas/semitons e distribuição por registo, sempre acompanhadas de contexto e indicação de estimativa.
- Distribuição de qualidade: takes bons, para rever e a repetir.
- Frequências de amostragem presentes e frequência de destino de exportação.
- Avisos de dataset: frequência mista, takes de ensaio sem áudio real, takes com pouca atividade vocal ou exportação vazia.

- Próxima ação recomendada: por exemplo, “rever 3 takes” ou “exportação pronta”.

Não apresentar um número de “qualidade do modelo” sem treinar e medir um modelo. O dashboard mede a completude e sinais observáveis do dataset, não promete o resultado do treino.

5.7 Exportação: pré-voo antes do download

Antes de gerar o ZIP, apresentar um resumo simples:

- Nome do dataset e quantidade de WAV.
- Formato, bit depth e frequência final.
- Duração de voz estimada.
- Avisos que não bloqueiam a exportação.
- Problemas que bloqueiam realmente a exportação, como ausência de takes válidos.
- Estimativa de tamanho e progresso quando disponível.
- Acesso separado a **Exportar para treino** e **Criar backup do projeto**.

Se houver uma frequência incompatível, não ignorar takes silenciosamente. Mostrar a lista afetada e permitir reamostrar, excluir de forma deliberada ou cancelar.

5.8 Design visual e sistema de componentes

Não faria um redesign só para parecer moderno. Consolidaria primeiro um sistema visual coerente:

- **Tipografia:** manter uma família de texto legível e uma fonte monoespaçada apenas para valores técnicos.
- **Hierarquia:** títulos curtos, corpo de texto confortável, métricas com unidades explícitas e rótulos consistentes.
- **Espaçamento:** escala de 4/8/12/16/24/32 px; usar os mesmos intervalos nos painéis e cartões.
- **Cores de estado:** sucesso, atenção e erro com contraste suficiente; nunca depender apenas da cor.
- **Botões:** primário, secundário e destrutivo; confirmar ações destrutivas e desativar ações realmente indisponíveis.
- **Cartões:** evitar excesso de caixas dentro de caixas. Usar separadores e espaço em branco para dividir áreas.
- **Feedback:** toast para confirmação breve; mensagem persistente junto ao controlo para erro que precisa de intervenção.
- **Estados vazios:** explicar o que falta e oferecer a ação seguinte, em vez de mostrar um painel em branco.

5.9 Responsividade e acessibilidade

A responsividade deve preservar as capacidades do produto em qualquer largura suportada. Não será aceite esconder análises, opções avançadas, gestão de takes ou exportação apenas porque o dispositivo é um telemóvel.

Critérios de layout e interação:

- Testar pelo menos 320, 360, 390, 430, 768, 1024 e 1440 px de largura, em retrato e paisagem quando aplicável.
- Manter o conteúdo utilizável sem scroll horizontal da página inteira; permitir scroll/zoom local em gráficos e waveform quando isso for necessário.
- Alvos tácteis com cerca de 44 × 44 CSS px como objetivo de desenho, separação suficiente entre ações destrutivas e ações comuns.
- Foco de teclado sempre visível; foco e ordem de leitura coerentes em desktop e mobile com teclado externo.
- Controlos avançados podem estar recolhidos, mas devem manter o mesmo valor, efeito e disponibilidade funcional em ambos os formatos.
- Todas as ações por hover/atalho de teclado devem ter alternativa visível e tátil; atalhos não podem ser o único caminho.
- Estados dinâmicos (gravação, erro, salvamento, interrupção) anunciados adequadamente a tecnologias de apoio, sem anunciar cada atualização do contador em excesso.
- Contraste de texto e controlos verificado segundo WCAG 2.2 AA como meta de projeto.
- Testar zoom, redução de movimento, leitores de ecrã e navegação por teclado; em mobile, testar VoiceOver e TalkBack nos fluxos críticos.
- Utilizar estados e rótulos textuais além da cor; não comunicar gravando/guardado/erro apenas através de uma cor ou ícone.

- Não depender de APIs que não existem uniformemente em navegadores móveis sem deteção de suporte, fallback e mensagem clara.

A checklist de regressão deve executar os mesmos fluxos de negócio nas variantes desktop e mobile. A UI pode reorganizar os controlos, mas não pode transformar o mobile numa versão funcionalmente reduzida.

5.10 Paridade funcional total em mobile — requisito obrigatório

Princípio não negociável: a versão mobile não é uma edição "lite", um companion app nem um gravador simplificado. Deve permitir executar os mesmos fluxos de trabalho e aceder às mesmas funções que o desktop. A adaptação responsiva altera a disposição, a navegação e o método de entrada; não altera a lista de funcionalidades disponíveis.

Matriz de paridade desktop–mobile

Área	Deve continuar disponível em mobile	Adaptação de interação permitida
Projetos e sessões	Criar, renomear, retomar, configurar e gerir projetos/sessões	Lista em cartões, menu contextual ou ecrã de detalhe
Perfil vocal	Iniciar/analisar, consultar resultados, editar manualmente intervalos e repetir testes	Gráficos responsivos, controlos tácteis e tabelas que possam expandir
Microfone e captura	Selecionar entrada quando o browser permitir, testar nível/ruído, configurar opções suportadas, gravar e interromper	Painel de entrada e controlos maiores; indicação explícita das opções que o browser não expõe
Exercícios e gravação	Aceder a todos os exercícios/packs, navegar, ver instruções, pré-contagem, guia de pitch e opções avançadas	Exercício e controlos em coluna; definições avançadas em acordeão, sem as remover
Análise e qualidade	Executar as mesmas análises, consultar todos os avisos e os seus detalhes	Métricas em cartões e gráficos com scroll/zoom local
Takes	Reproduzir, comparar quando disponível, renomear, aceitar, repetir, eliminar, inspecionar estado e persistência	Lista de cartões; ações secundárias no menu, mantendo as ações principais visíveis
Dataset/dashboard	Ver os mesmos contadores, cobertura, distribuição, filtros, avisos e estados	Blocos empilhados e gráficos adaptados à largura; tabelas com vista detalhada
Exportação de treino	Mesmos formatos/opções suportados, seleção de takes, configuração de sample rate, manifesto, pré-voo, progresso e cancelamento	Passos verticais e resumo antes de iniciar; feedback de download específico do SO/browser
Backup e restauro	Criar, importar, validar, pré-visualizar, mesclar/substituir quando suportado e recuperar backup	Seletor de ficheiros e confirmação adaptados ao sistema operativo
Definições e acessibilidade	Todas as definições funcionais, idioma, calibração, atalhos e preferências disponíveis	Navegação em secções; atalhos de teclado podem ter equivalentes tácteis, nunca ser o único modo

Esta matriz deve transformar-se num inventário de todas as ações atualmente existentes na aplicação. Antes do desenvolvimento, criar uma **matriz real de paridade** que enumere cada botão, ação de menu, parâmetro e estado de desktop, indique o equivalente mobile e associe um teste. Não assumir cobertura só porque uma página fica visualmente responsiva.

Layout e navegação mobile

1. **Manter todas as cinco áreas** — Visão geral, Perfil vocal, Estúdio, Takes e Dataset/Exportação — acessíveis numa navegação previsível. Usar barra inferior, menu ou separadores conforme o espaço, mas nunca eliminar destinos existentes.

2. Em retrato, empilhar painéis e permitir abrir a análise completa num detalhe; em paisagem/tablet, recuperar colunas quando houver espaço. Não reduzir a precisão dos controlos apenas para caber.
3. Manter disponíveis os parâmetros avançados. Podem estar recolhidos por defeito, mas têm de ser localizáveis, legíveis e editáveis em mobile.
4. Oferecer ações tácteis para todas as operações que no desktop tenham atalho de teclado ou hover. Gestos são complementares e não podem ser o único caminho.
5. Garantir zoom local de waveform/gráficos e rolagem dentro de painéis sem causar scroll horizontal da página inteira.
6. Preservar seleção, posição no exercício, filtros e estado do painel ao mudar de área ou orientação, sempre que tecnicamente possível.

Microfone móvel como entrada de primeira classe

O motivo para suportar mobile não é apenas conveniência. Alguns telemóveis atuais têm microfones e cadeias de captura muito competentes e podem produzir gravações melhores do que um computador com microfone integrado de qualidade inferior. Isso não é universal: o resultado depende do modelo, da posição, do ambiente, do processamento do sistema e do browser. A aplicação deve medir e permitir comparar, não presumir que desktop ou mobile é sempre superior.

Implementação recomendada:

1. Enumerar e apresentar as entradas disponíveis quando o browser permite; explicar que a lista pode ficar limitada até a permissão ser concedida.
2. Permitir selecionar uma entrada quando suportado e mostrar a entrada efetivamente selecionada, não apenas a preferência pedida.
3. Expor opções de captura como cancelamento de eco, supressão de ruído e controlo automático de ganho quando o browser permitir configurá-las. Mostrar “suportado”, “não suportado” ou “estado desconhecido”; não fingir que uma preferência foi aplicada.
4. Quando disponível, consultar as definições reais da faixa de áudio (por exemplo, canal e frequência de amostragem) e informar o utilizador. Distinguir frequência capturada de frequência final exportada.
5. Oferecer teste A/B de nível/ruído e uma gravação de teste reproduzível para comparar entradas. Não classificar um microfone apenas pela marca ou pelo tipo de dispositivo.
6. Dar instruções específicas para telemóvel: distância da boca, orientação, posição para evitar fricção/vento, capa que tapa microfones e escolha entre microfone integrado, auricular com fios ou Bluetooth quando disponíveis.
7. Nunca ativar processamento adicional silenciosamente se isso puder alterar a voz. Mostrar as opções efetivamente ativas e documentar diferenças entre browser/sistema operativo.

Interrupções e limites do sistema operativo

A paridade significa que todos os recursos existem na UI mobile; não significa que o browser permita executar uma captura contínua em qualquer circunstância. Chamadas, bloqueio do ecrã, troca de aplicação, Bluetooth, economia de energia ou suspensão da página podem interromper áudio, especialmente em iOS e em browsers móveis.

- Escutar eventos de ciclo de vida e interrupções relevantes; quando a captura for interrompida, mudar imediatamente o estado para “gravação interrompida” e preservar o segmento capturado se for válido.
- Não prometer gravação em background quando a plataforma não a suporta. Explicar a limitação antes da sessão e oferecer a melhor recuperação possível.
- Ao regressar à aplicação, não retomar gravação automaticamente sem indicação/consentimento claro.
- Testar perda e reconexão de Bluetooth, chamadas simuladas, mudança de app, ecrã bloqueado e retoma. Documentar os limites de cada navegador suportado.
- Se um browser não disponibilizar uma capacidade específica (por exemplo, escolher a entrada ou escrever diretamente para ficheiro), manter a função através de uma alternativa viável quando possível ou explicar precisamente a limitação da plataforma. Não esconder nem remover a função sem aviso.

Critérios de aceitação para paridade

- Cada ação funcional do desktop está representada no inventário de paridade e tem uma forma equivalente de utilização em mobile.
- Não há fluxos cuja conclusão dependa exclusivamente de hover, rato, teclado físico ou de um viewport largo.
- Em telemóvel é possível completar, sem ajuda do desktop: criar/abrir projeto, configurar entrada, executar perfil vocal, concluir exercícios, gravar/rever/aceitar takes, gerir o dataset, ajustar as mesmas opções disponíveis, exportar e restaurar backup.
- A aplicação não declara “guardado” sem confirmação; interrupções e falhas deixam um estado explícito.
- A mesma lógica de negócio e as mesmas regras de validação são usadas em desktop e mobile; só os adaptadores de interação/layout variam.
- Testes de regressão cobrem todas as funções críticas em viewport mobile e em pelo menos um dispositivo Android e um iPhone/iPad real antes de declarar suporte estável.

5.11 Microcopy: exemplo de mensagens melhores

Situação	Texto proposto
Microfone bloqueado	“O browser não permitiu o acesso ao microfone. Autorize o microfone nas definições do site e tente novamente.”
Nível baixo	“O sinal está baixo. Aproxime-se um pouco do microfone ou aumente ligeiramente o ganho e repita o teste.”
Possível clipping	“O sinal está a atingir o limite digital. Reduza o ganho antes de gravar os takes seguintes.”
A guardar	“A guardar este take neste navegador…”
Falha de persistência	“O take está nesta sessão, mas não foi guardado de forma duradoura. Tente novamente ou exporte-o já.”
Exportação a decorrer	“A preparar 18 de 42 ficheiros. Pode cancelar sem apagar as gravações.”
Exportação concluída	“Dataset exportado. O ficheiro contém 42 WAV e o manifesto de configuração.”
Armazenamento persistente indisponível	“O navegador não confirmou armazenamento persistente. Exporte um backup antes de fechar a sessão.”

6. Método de implementação

Ciclo de trabalho recomendado

Para cada alteração, repetir o mesmo ciclo curto:

1. **Abrir uma issue** com problema, contexto, risco e critérios de aceitação.
2. **Reproduzir ou demonstrar a falha** com um teste antes de alterar o comportamento.
3. **Implementar a menor alteração completa** dentro do módulo responsável.
4. **Adicionar testes unitários** quando a regra é determinística; acrescentar teste de browser quando depende de permissões, foco, navegação ou recursos reais.
5. Executar npm run check e rever os resultados.
6. Fazer revisão manual da UI em largura desktop e mobile; percorrer a matriz de paridade; testar toque, teclado, interrupções de áudio e estados de erro.
7. Atualizar a documentação técnica e os critérios de suporte.
8. Criar release só depois da lista de aceitação estar concluída.

Estratégia de branches e PRs

- Um ramo por alteração: fix/sample-rate-manifest, fix/persist-take-state, perf/streaming-export, test/browser-recording-flow, feat/dataset-dashboard.
- Um PR deve resolver um objetivo principal, em vez de juntar backend de exportação, redesign e localização num bloco único.
- A CI deve continuar a executar lint, typecheck, validação de conteúdo, testes e build.
- Não misturar refatoração cosmética com correções de dados de alto risco.
- Para mudanças que alterem o formato de backup, versionar explicitamente o formato e testar compatibilidade entre versões.

---

7. Cronograma proposto — oito semanas

Premissa de planeamento: um programador, trabalho parcial, aproximadamente 8–12 horas por semana. É uma ordem de execução, não um prazo garantido. A dimensão real deve ser revista depois do primeiro baseline e da medição da exportação.

Semana	Foco	Entregas concretas	Critério para avançar
0 — Baseline	Reproduzir e medir	npm ci, npm run check, dataset de referência, verificação de headers WAV/JSON, registo do fluxo atual e capturas de UI desktop/mobile	Estado atual documentado; falhas reproduzíveis e lista de navegadores-alvo definida
1 — Integridade do áudio	P0: sample rate	Política de frequência; manifesto coerente; testes WAV/JSON a 40k, 44,1k e 48k; decisão explícita para takes mistos	Não existe divergência silenciosa entre áudio exportado e manifesto
2 — Persistência e recuperação	P0: garantir takes	Estados de save; retry; erros de IndexedDB; try/finally em medição/teste de microfone; testes com falhas simuladas	Um take nunca aparece como guardado sem confirmação; operações falhadas deixam UI utilizável
3 — Exportação escalável	P1: memória	Medições antes/depois; encoder por blocos; ZIP progressivo; cancelamento e feedback de progresso	ZIP verificado e sessão preservada quando a exportação falha/cancela
4 — Backup e compatibilidade	P1: continuidade	Backup/restauro versionado; teste de dados corrompidos; matriz inicial de navegadores e guia de teste	Backup de amostra restaura corretamente e erros de importação são seguros
5 — UX e paridade funcional	Desktop + mobile	Matriz completa de paridade; layout adaptativo; controlos tácteis; fluxos completos de perfil vocal, gravação, takes e definições em mobile	Todas as ações desktop têm equivalente mobile documentado e testado; nenhuma funcionalidade retirada por viewport
6 — Dataset, exportação e dispositivos	Paridade + usabilidade	Dashboard e pré-voo responsivos; mesmos filtros/opções; teste de entradas de áudio; testes em Android e iOS reais; recuperação de interrupções	É possível concluir o fluxo completo no telemóvel e os limites do SO são explícitos; exportações/backup funcionam ou oferecem fallback compreensível
7 — Acessibilidade, paridade e release	P2/P3	WCAG, VoiceOver/TalkBack, regressão completa desktop/mobile, contraste, documentação de compatibilidade, screenshot real, vídeo, changelog e release	Matriz de paridade a 100% para as funções acordadas; navegadores/dispositivos suportados documentados com resultados reais

Se só houver tempo para uma primeira sprint curta

Executar primeiro as semanas 0–2, concentrando o âmbito em: (1) frequência/manifesto, (2) falha de persistência, (3) recuperação de estados assíncronos. São alterações menos vistosas do que um redesign, mas protegem o valor central da aplicação: os takes que o utilizador grava.

---

8. Plano de testes e critérios de aceitação

Área	Cenários mínimos	Resultado esperado
Exportação áudio	40k, 44,1k e 48k; 16/24-bit se suportado; mono; take vazio ou inválido	Cabeçalho WAV, conteúdo e JSON coerentes; erro compreensível para entradas inválidas
ZIP	Poucos takes, muitos takes, cancelamento, falha de encoding e verificação do arquivo	ZIP válido, sem ficheiros inesperadamente omitidos; sessão original intacta
Persistência	IndexedDB normal, indisponível, quota/falha simulada, reload após guardar	Estado de guardar correto; retry disponível; recuperação de takes confirmados
Microfone	Permissão concedida/recusada, sem dispositivo, dispositivo removido, captura interrompida	Estado regressa a um ponto utilizável e a mensagem indica o passo seguinte
Fluxo UX	Gravar → ouvir → repetir → aceitar → avançar → exportar	Sem ambiguidades entre aceitar e guardar; atalhos não disparam em inputs
Offline/PWA	Primeira visita online e visita seguinte offline, quando aplicável	Recursos estáticos carregam offline; a UI comunica limitações sem rede se existirem
Acessibilidade	Teclado, foco, zoom, contraste, mensagens dinâmicas, redução de movimento, VoiceOver/TalkBack	Ações críticas são executáveis sem rato e estados são percetíveis sem depender apenas da cor
Paridade desktop/mobile	Inventário de cada ação, definição e fluxo; retrato/paisagem; Android e iOS reais	100% das funções incluídas no âmbito têm equivalente mobile testado; nenhum fluxo crítico exige desktop
Entrada de áudio mobile	Microfone integrado, acessórios disponíveis, permissões, settings reais, Bluetooth, interrupções e retorno à aplicação	A entrada efetiva é clara, a captura é avaliada por sinal e falhas/interrupções preservam dados válidos
Localização	Inglês, pt-BR, pt-PT, espanhol e chinês suportados pelo produto	Sem strings em falta, cortes críticos ou mistura acidental de idiomas nos fluxos principais

Gate de release

Não publicar uma versão como estável sem:

- npm run check verde.
- Exportação de referência validada por um verificador WAV/ZIP.
- Teste de persistência e recuperação após reload.
- Teste manual do fluxo de gravação e de erro do microfone nos navegadores oficialmente listados.
- Confirmação de que o README não promete comportamento que a build publicada não possui.

---

9. Métricas para avaliar se as mudanças funcionaram

Evitar métricas de vaidade; medir sucesso real do fluxo. Os valores-alvo devem ser estabelecidos após o baseline.

- **Integridade do pacote:** percentagem de exportações de teste em que cabeçalho WAV e metadados correspondem; objetivo de aceitação: 100% nos cenários automatizados.
- **Persistência:** percentagem de takes aceites que ficam confirmados em IndexedDB; objetivo de aceitação: 100% em ambiente normal e falha explicitamente apresentada nos cenários simulados.
- **Recuperação:** percentagem de takes confirmados que reaparecem após reload; objetivo de aceitação: 100% em ambiente normal de teste.
- **Exportação:** tempo e pico de memória para datasets de referência; publicar os valores medidos, não estimativas.
- **Taxa de falha:** exportações com erro, capturas interrompidas e falhas de restauro durante a bateria de testes.
- **Usabilidade:** número de erros críticos numa sessão de teste, tempo para concluir o primeiro take e percentagem de utilizadores que identificam corretamente o estado de gravação/guarda.
- **Acessibilidade:** defeitos críticos encontrados no teste manual e nas ferramentas automatizadas.
- **Paridade funcional mobile:** percentagem de ações desktop com equivalente mobile validado; meta de release: 100% das funções acordadas.
- **Qualidade de captura por dispositivo:** comparar medições de sinal e feedback em aparelhos de referência; não assumir superioridade de uma classe de dispositivo sem dados.

Como a aplicação defende privacidade local, não introduziria tracking remoto por defeito para recolher estas métricas. A avaliação inicial pode ser feita com testes automatizados, medições manuais e feedback voluntário.

---

10. Riscos e decisões a evitar

Riscos principais

- **Reamostragem mal implementada:** pode alterar o áudio ou introduzir artefactos. Usar um método conhecido, validar por sinais de teste e comparar espectro/duração.
- **Exportação incremental complexa:** requer atenção a buffers, encerramento de ZIP e cancelamento. Implementar depois de medir o custo da solução atual.
- **Backup incompatível entre versões:** incluir versão explícita, validação e estratégia de migração.
- **Dashboard enganador:** cobertura de notas/registos é uma métrica de dataset, não garantia de qualidade do modelo treinado.
- **Excesso de UI:** muitos indicadores, cartões e gráficos podem distrair durante a gravação. Priorizar o que ajuda a decidir a próxima ação.
- **“Mobile-lite” por acidente:** esconder funções avançadas ou retirar opções para simplificar o layout rompe o requisito de paridade. Reorganizar os mesmos controlos e validar pela matriz.
- **Variação de entrada de áudio:** a qualidade depende do modelo, posicionamento, processamento do SO/browser e ambiente. Oferecer seleção/diagnóstico quando suportados e avaliar sinal em vez de presumir que telemóveis ou computadores são melhores.
- **Recomendações vocais rígidas:** apresentar intervalos e distribuições como orientação empírica, ajustável pelo utilizador, e não como regras universais.

O que eu não faria agora

- Não migraria de Preact para React sem um problema mensurável que justifique o custo.
- Não introduziria um backend, contas ou sincronização cloud num produto cujo diferencial é a privacidade local.
- Não acrescentaria telemetria remota como condição para usar a aplicação.
- Não adicionaria dezenas de gráficos antes de melhorar persistência e exportação.
- Não afirmaria que um dataset é “pronto para treino” sem definir e verificar os critérios concretos por trás desse estado.
- Não reescreveria o algoritmo de pitch sem benchmarks que demonstrem um problema real.

11. README, documentação e presença pública

A documentação pública deve ajudar dois públicos diferentes: quem só quer gravar/exportar e quem quer contribuir para o código.

Estrutura recomendada para o topo do README

1. Nome, frase de valor e screenshot real da aplicação.
2. Três benefícios concretos: gravação guiada, revisão de qualidade e exportação local.
3. Botão/link para experimentar a aplicação.
4. Compatibilidade verificada e aviso sobre permissões de microfone.
5. Fluxo ilustrado em três passos.
6. Instalação rápida para desenvolvimento.
7. Estado do projeto, limitações conhecidas e links para guias técnicos.
8. Privacidade e uso responsável da voz.

Ações de documentação

- Corrigir o uso de SOVPE para SOVT.
- Clarificar os níveis de ruído: ideal, aceitável e motivo para repetir; explicar que o resultado depende do microfone e da sala.
- Separar os valores efetivos da exportação das sugestões para treinar no Applio/RVC.
- Incluir a política de frequências e a forma como takes incompatíveis são tratados.
- Adicionar uma captura verdadeira de cada etapa principal e um vídeo de 20–40 segundos.
- Publicar uma secção “Limitações conhecidas” e um changelog por release.
- Indicar apenas os browsers e versões realmente testados.
- Manter guias técnicos extensos em docs/, deixando o primeiro ecrã do README mais direto.

12. Backlog inicial de issues

A lista seguinte pode ser transferida diretamente para GitHub Issues/Projects.

Issue sugerida	Prioridade	Dependências	Entrega
fix: align sample-rate metadata with exported WAV	P0	Baseline de exportação	Frequências coerentes e testes de headers
fix: model persistent-save state for accepted takes	P0	Nenhuma	Estado saved/unsaved e retry
fix: guarantee cleanup for mic/noise checks	P0	Nenhuma	try/catch/finally e testes de falha
feat: export WAV/ZIP incrementally	P1	Baseline de memória; T-01	Menos buffers duplicados, progresso e cancelamento
feat: add project backup and restore	P1	Esquema de backup	Backup versionado, validação e preview de importação
test: add browser end-to-end recording flow	P1	Testes unitários estáveis	Permissões, gravação, revisão, persistência e exportação
feat: explain take-quality warnings	P2	T-03	Alertas acionáveis e estados claros
feat: add dataset coverage and export preflight	P2	T-01, T-02	Dashboard e validação pré-exportação
ux: polish onboarding and recording workspace	P2	Feedback de utilizadores	Fluxo simplificado e consistente
docs: refresh README and add real workflow demo	P3	Fluxo estabilizado	Screenshot, vídeo, limitações e releases coerentes

13. Definição de sucesso do plano

O projeto terá evoluído de forma material quando um utilizador conseguir:

1. Abrir a aplicação e perceber imediatamente como começar.
2. Preparar o microfone sem adivinhar qual é o problema quando algo falha.
3. Gravar um take, ouvi-lo, compreender os alertas e decidir se o aceita.
4. Saber com certeza se o take foi guardado localmente.
5. Fechar e reabrir a aplicação sem perder os takes confirmados, no ambiente de armazenamento normal.
6. Ver a cobertura e os problemas do dataset antes de exportar.
7. Exportar WAV/ZIP com metadados coerentes, sem perder takes silenciosamente.
8. Criar e restaurar um backup de projeto separado do pacote final de treino.
9. Usar todas as funções do desktop num ecrã estreito, com controlos tácteis e layout adaptado.
10. Selecionar/testar entradas de áudio móveis quando o browser as disponibilizar e compreender interrupções ou limitações impostas pelo sistema operativo.

Decisão recomendada: iniciar pela correção de frequência/manifesto, seguida do estado de persistência e da recuperação de erros. Estes três itens protegem a confiança nos dados. Depois, tratar a paridade funcional desktop/mobile como critério de aceitação de toda a UI: mesmo produto e mesmos fluxos em ambos, com layout adaptado e suporte de áudio móvel testado em aparelhos reais.

14. Fontes técnicas consultadas

- Repositório principal
- README principal — funcionalidades declaradas, fluxo de trabalho, parâmetros e documentação pública.
- package.json — scripts, dependências e npm run check.
- src/app/controller.ts — persistência, captura, metadados e exportação e secção do manifesto/exportação.
- src/export/dataset.ts — codificação de WAV e montagem do ZIP.
- src/storage/idb.ts — IndexedDB e persistência solicitada.

Nota final: este plano é uma proposta técnica priorizada, não um relatório de execução. Cada alteração deve ser confirmada por testes e pela aplicação em funcionamento antes de ser considerada concluída.